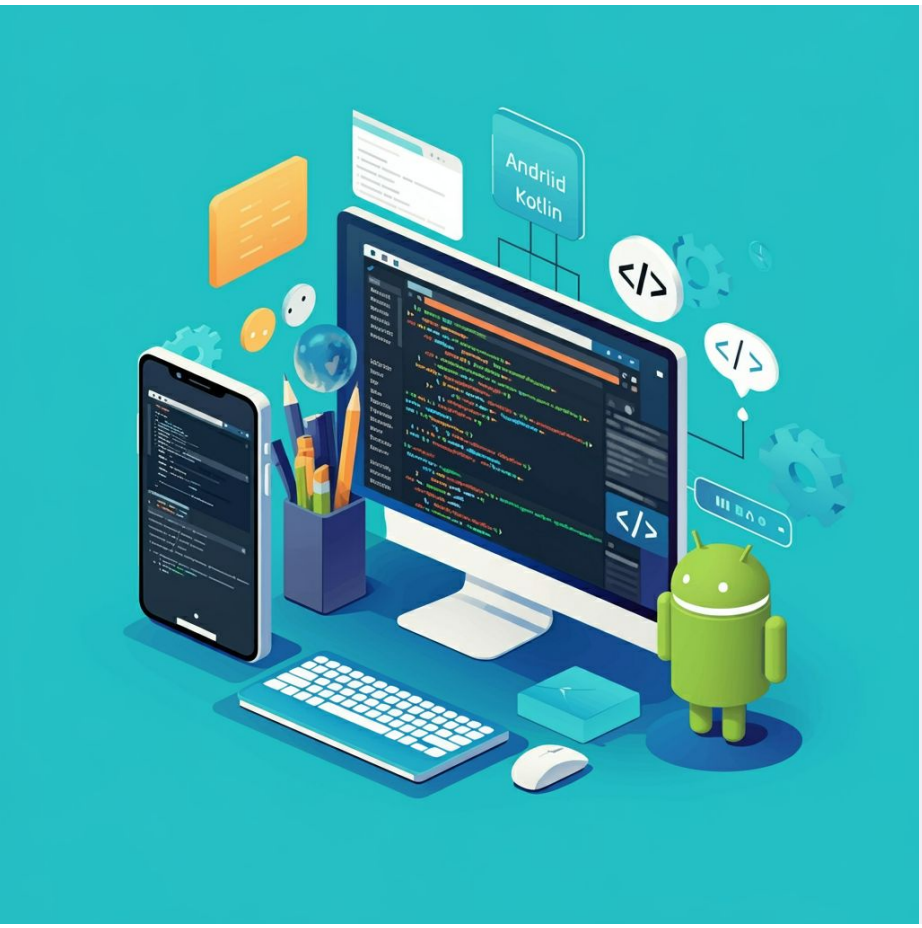


Aula 3: Conectando Telas e Dados no Android



TRANSIÇÕES ENTRE ACTIVITIES

Em aplicativos diversos fazemos transições entre telas que carregam funcionalidades específicas.

Conectando Telas e Dados no Android

Expandindo Horizontes: Navegando entre Activities

- Até agora, trabalhamos com uma única tela.
- A maioria dos aplicativos possui múltiplas telas interconectadas.
- Vamos aprender a navegar entre diferentes telas (Activities) no Android.
- Apresentaremos o conceito de Intents e como usá-los para comunicação entre Activities.

Intents: A Ponte Entre Activities

Intents: Mensageiros do Android

- **Definição**: Objetos que permitem a comunicação entre componentes do Android.

- **Usos**:
  - Iniciar novas Activities.
  - Passar dados entre Activities.
  - Iniciar serviços.
  - Transmitir broadcasts para o sistema.

Iniciando uma Nova Activity

Comunicação Direta: Iniciando uma Activity com Intent

- **Método**: startActivity(intent)

- **Exemplo**:

  val intent = Intent(this, SegundaActivity::class.java)

  startActivity(intent)

Passando Dados com Intents

Compartilhando Informações: Enviando Dados entre Activities

Método: putExtra(chave: String, valor: Any)

Exemplo:

```
// Na Activity de origem:
intent.putExtra("nome_usuario", usuario.nome)

// Na Activity de destino:
val nomeUsuario = intent.getStringExtra("nome_usuario")
```

- Observação: Utilize a mesma chave para enviar e recuperar o dado.

+ACTIVITY

- Cada Activity pode iniciar outras Activities para executar tarefas diferentes.

- Quando isso acontece, a Activity que foi iniciada é empilhada sobre a principal.

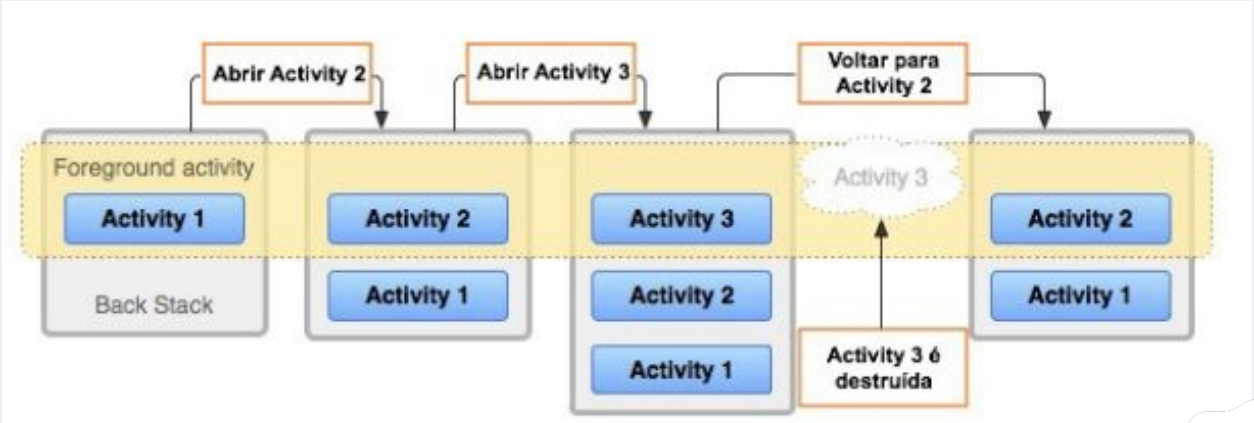


Figura 4.1: Pilha de navegação Activity.

MUITAS ACTIVITIES

O grande problema dessa forma, você já deve imaginar, é que agora teríamos duas Activities na pilha de navegação! Isso é um problema porque a memória RAM do aparelho ficaria se enchendo de "lixo" – "lixo" porque a pilha de navegação só cresceria.



ACTIVITY CODE - ESTADOS

```
import android.app.Activity
import android.os.Bundle

class MainActivity : Activity(){
        override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        }
}
```



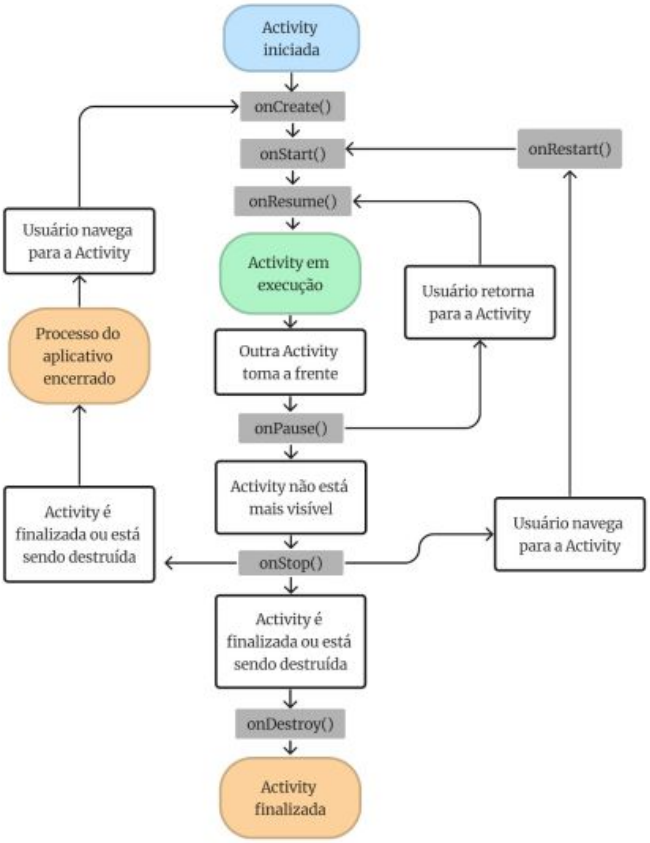


Figura 4.2: Ciclo de vida de uma Activity.



ACTIVITY CONTENT

Definir o **conteúdo** de uma Activity significa definir o conteúdo que aparecerá na tela, para isso o Android possui o método **setContentView(*Conteúdo*).**

Devemos passar o conteúdo que vamos montar na tela para ele. Existem essencialmente duas formas de passar esse conteúdo:



ACTIVITY CONTENT

A primeira é criando todo o layout da tela com código em Kotlin e passá-lo no método setContentView .

- Só conseguiria ver o resultado ao compilar e rodar a aplicação.
- Layout fica misturado ao código relacionado à lógica
- Em jogos, a tela é constantemente atualizada, a todo momento há objetos sendo criados e retirados da tela.



```
import android.app.Activity
import android.os.Bundle
import android.widget.TextView

class MainActivity : Activity() {
        override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        //Criando um objeto de texto
        val texto = TextView(this)
        texto.text = "Hello Kotlin"
        //definindo o conteúdo da tela
        setContentView(texto)
        }
}
```



ACTIVITY CONTENT

A outra forma de criação de layout, e a mais comum, é utilizar um arquivo xml e indicá-lo no método setContentView . res/layout/activity_main.xml

- Separar o layout da tela em um arquivo xml, responsabilidades ficam separadas
- Pode ser reutilizado em alguma outra Activity
- podemos utilizar a janela de Preview (pré-visualização) do Android Studio;



ANDROIDX

Diferenças entre **Activity** e **AppCompatActivity**

androidx.appcompat.app.appCompatActivity
Esse pacote identifica que a classe não faz parte do SDK do Android, mas sim de uma biblioteca externa.

AndroidX faz parte do conjunto de bibliotecas do **Android Jetpack**

Permite retrocompatibilidade!



ANDROIDX

```
class MainActivity : Activity( ) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
}

class MainActivity: AppCompatActivity(R.layout.activity_main) {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Código omitido
}
```



CRIANDO ACTIVITY

Botão Direito do Mouse Sobre o Projeto Crie uma New Activity.

A IDE já implementa os recursos necessário para desenvolvimento:

- Arquivo xml Layout separado
- Opções de Classe padrão de Lançamento



FLOAT ACTION BUTTON

Adicionaremos o Floating Action Button (botão de ação flutuante) ao layout da activity_main.xml utilizando os componentes do Material Design. Para isso, basta pesquisá-lo na aba de pesquisa, clicar sobre ele e arrastá-lo para a tela.

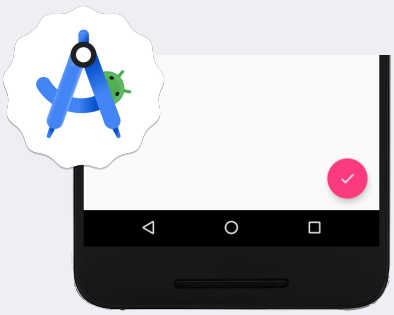
- Vector Asset cria apenas um único recurso que se ajusta automaticamente ao tamanho da tela e à densidade de pixels (DPI)



FLOAT ACTION BUTTON

```
val fab =findViewById<FloatingActionButton>(R.id.floatingActionButton)

fab.setOnClickListener {
    val intent = Intent(this, FormularioProdutoActivity::class.java)
    startActivity(intent)
}
```



FLOAT ACTION BUTTON

```
val btnVoltar =findViewById<FloatingActionButton>(R.id.btn_voltar)

btnVoltar.setOnClickListener {
        //Código
        finish()
}
```

